Testing Ajax Calls

Now that you have the internal logic of your task list working, let's focus on how to get data into and out of this code. The previous version of the code read the tasks essentially from the DOM, but in this case you'll make a separate Ajax call to get that information from the server. Let's talk about strategies to test Ajax calls.

Here's the basic structure of an Ajax call—this one uses jQuery, but the basic idea holds no matter how the call gets made:

```
$.ajax({
  url: `projects/${this.id}.js`
  dataType: "json"
}).then((data) => {
  this.name = data.project.name
  // and so on
})
```

The jQuery $.ajax method takes the URL and other information about the call you want to make and returns a JavaScript promise object. (A decent starting point on what promises are can be found online.[6]) Chain the then method to that promise, and the argument to then needs to be a function. That function will be executed once the server call has completed. The promise object lets you manage the asynchronous nature of the server call because the function passed to then is not executed until after the server call has completed.

This $.ajax pattern has a lot of issues for us as testers:

- The external server targeted by the $.ajax call is a dependency outside the realm of your JavaScript unit-test runtime, although it is possible to call it inside the Capybara tests I discussed in the last chapter.
- The asynchronous nature of the call makes it challenging to test for the effects that are supposed to happen after the Ajax call—you just don't know how long to wait for those effects to happen.

Both of these problems are slightly exacerbated by the way the previous code intertwines three things: the Ajax call, the callback, and the structure that relates them. What happens if you write it more like this?

6. https://developers.google.com/web/fundamentals/getting-started/primers/promises